

Claude Code Task Brief

Offline .NET Code Reviewer with Ollama + Qwen2.5-Coder 7B

Target	Portfolio-ready GitHub MVP
Timebox	Approximately 60 minutes
Platform	.NET console application
LLM	Qwen2.5-Coder 7B via local Ollama
Key message	Privacy-first AI code review: source code stays on the machine

1. Goal

Build a small but polished .NET console application that reviews a local C# source file using Qwen2.5-Coder 7B running through Ollama. The application must work locally and must not require a cloud LLM or API key. The project should demonstrate a practical business use case for private/offline AI.

Primary demo

A developer passes the path of a .cs file. The program reads the file, asks the local coding model to review it, and prints a structured review containing issues, severity, explanations, and suggested fixes.

Example invocation

```
dotnet run -- "Samples/BadCustomerService.cs"
```

2. Hard Scope - Keep This to One Hour

Do not add a web UI, database, authentication, RAG, vector database, Docker, MCP server, multi-agent workflow, cloud deployment, or repository-wide autonomous editing. Prefer a finished, readable MVP over extra features.

Required features:

1. Accept a path to one C# file.
2. Validate that the file exists and is a .cs file.
3. Read its source code locally.
4. Send the code to local Ollama using model **qwen2.5-coder:7b**.
5. Ask for a structured code review.
6. Parse the model response into C# objects when possible.
7. Print a clean console report.
8. Include one deliberately flawed sample C# file for the demo.

3. Local Architecture

.NET Console App -> Ollama local API -> Qwen2.5-Coder 7B

Ollama endpoint: `http://localhost:11434`

No OpenAI/Azure/Anthropic API key should be required.

Use Ollama's local HTTP API directly or an appropriate lightweight .NET client/library if it clearly reduces implementation time. Avoid unnecessary abstractions.

4. Suggested Project Structure

```
OfflineDotNetCodeReviewer/ |-- Program.cs |-- Services/ | |-- OllamaCodeReviewService.cs | `--  
ICodeReviewService.cs |-- Models/ | |-- CodeReviewResult.cs | `-- CodeIssue.cs |-- Prompts/ | `--
```

```
CodeReviewPrompt.cs |-- Samples/ | `-- BadCustomerService.cs |-- README.md `-- .gitignore
```

5. Structured Result

Use a simple result model similar to:

```
public sealed record CodeReviewResult( string Summary, List<CodeIssue> Issues); public sealed
record CodeIssue( string Category, string Severity, string Explanation, string SuggestedFix);
```

Suggested categories: Bug, Security, Code Smell, Performance, Maintainability, Testing. Suggested severity values: Low, Medium, High, Critical.

6. Prompt Requirements

The system/review prompt should tell the model that it is reviewing C#/.NET code. It should inspect only the provided source, avoid inventing missing context, clearly say when something cannot be determined, prioritize real defects over stylistic nitpicks, and return JSON matching the requested schema.

Ask it to focus on: correctness/bugs, security, null handling, exception handling, performance, maintainability, and obvious missing test cases. Keep recommendations concise and actionable.

7. Safety / Privacy Behavior

The application must only read the file explicitly supplied by the user. It must not upload source code anywhere. Do not implement arbitrary shell execution or file modification in this MVP. If Ollama is unavailable, print a useful message telling the user to start Ollama and verify that qwen2.5-coder:7b is installed.

8. Demo Sample

Create a small BadCustomerService.cs containing several easy-to-detect problems, for example: possible null dereference, swallowed exception, hard-coded secret-like value, inefficient repeated enumeration, and insufficient input validation. Keep it realistic and short enough that the review completes quickly on CPU.

9. Console Output

```
OFFLINE .NET CODE REVIEW Model: qwen2.5-coder:7b File: BadCustomerService.cs Summary: ... Issues:
4 [HIGH] Security Explanation: ... Suggested fix: ... [MEDIUM] Bug Explanation: ... Suggested
fix: ...
```

Plain text output is sufficient. Use clean formatting; do not spend the timebox building a rich terminal UI.

10. README Requirements

The README is important for the GitHub portfolio. Include: project purpose, privacy/offline value proposition, architecture, prerequisites, Ollama/model setup, how to run the demo, example output, limitations, and a short 'Why local AI?' section.

Suggested README headline:

Offline .NET Code Reviewer - Privacy-first AI code review with .NET, Ollama and Qwen2.5-Coder. Source code is analyzed locally and never needs to be sent to a cloud LLM.

11. One-Hour Execution Plan

0-10 min	Create project, models and Ollama HTTP call.
10-25 min	Implement prompt + review service + JSON parsing.
25-35 min	Implement CLI input/validation and formatted output.
35-45 min	Create flawed sample file and test end-to-end.

45-60 min

Clean code, error handling, README and final GitHub polish.

12. Definition of Done

The project is complete when a fresh user who already has Ollama and qwen2.5-coder:7b installed can clone the repository, run the application against the included sample .cs file, receive a useful structured review, and understand from the README that inference is performed locally.

Do not expand the scope after this point. If time remains, improve the README or error handling rather than adding another framework.

Optional stretch goal (only if ahead of schedule)

Allow the user to pass a second .cs file or choose between two review modes: Quick Review and Security Review. Do not implement autonomous code changes.